

Дипломный проект «Построение рекомендательной системы для интернет магазина».

Курс: «Профессия Data Science»

Трек: «ML - Инженер»

Поток: DSPR_143 (131)

Студент: Варламов Александр Владимирович

Москва, Март 2025.

Задачи и цели проекта

****Компания ритейл (заказчик)**** - поставил задачу повысить прибыль от дополнительных продаж в интернет-магазине, не менее чем на 20%, за счет внедрения рекомендательной системы, которая должна выводить 3 товара на главной странице сайта (клиента).

- **Бизнес задача:** повышение прибыли от дополнительных продаж в интернет магазине не менее чем на 20% от внедрения рекомендательной системы на главной странице сайта.
- **Техническая задача:** разработать и внедрить рекомендательную систему на главной странице сайта, которая бы рекомендовала пользователям не менее 3-х товаров.
- **Цели проекта:**
 1. Постановка задачи - формирование бизнес и технической задачи проекта, формирование технической метрик качества;
 2. Исследование данных - получение и описание структуры данных, статистический анализ данных;
 3. Создание факторов для модели - связанных с клиентами и товарами;
 4. Проведение экспериментов - коллаборативная фильтрация, факторизационные машины, XGBoost для задач классификации;
 5. Создание MVP - проектирование API сервиса, создание web-сервиса с моделью;
 6. Контейнеризация - создание Docker - контейнера с сервисом.

Метрика качества: заказчиком принято решение производить оценку работы моделей с помощью метрики **Precision@3** – точность по первым 3 рекомендациям.

Структура данных

Набор данных состоит из трех файлов: файла с данными о поведении (events.csv), файлы со свойствами элемента (item_properties.csv: представлен 2 файлами) и файл, описывающий дерево категорий (category_tree.csv). Данные были собраны с реального веб-сайта электронной коммерции. Это необработанные данные, то есть без каких-либо преобразований содержимого, однако все значения хэшируются из-за проблем с конфиденциальностью.

1. Данные о поведении, то есть такие события, как клики, добавления в корзины, транзакции, представляют собой взаимодействия, которые были собраны за период в 4,5 месяца. Посетитель может совершать три типа событий, а именно "просмотр", "добавление" или "транзакция". Всего зарегистрировано 2 756 101 событие, включая 2 664 312 просмотров, 69 332 добавления в корзину и 22 457 транзакций, произведенных 1 407 580 уникальными посетителями. Примерно для 90% событий соответствующие свойства можно найти в файле со свойствами.

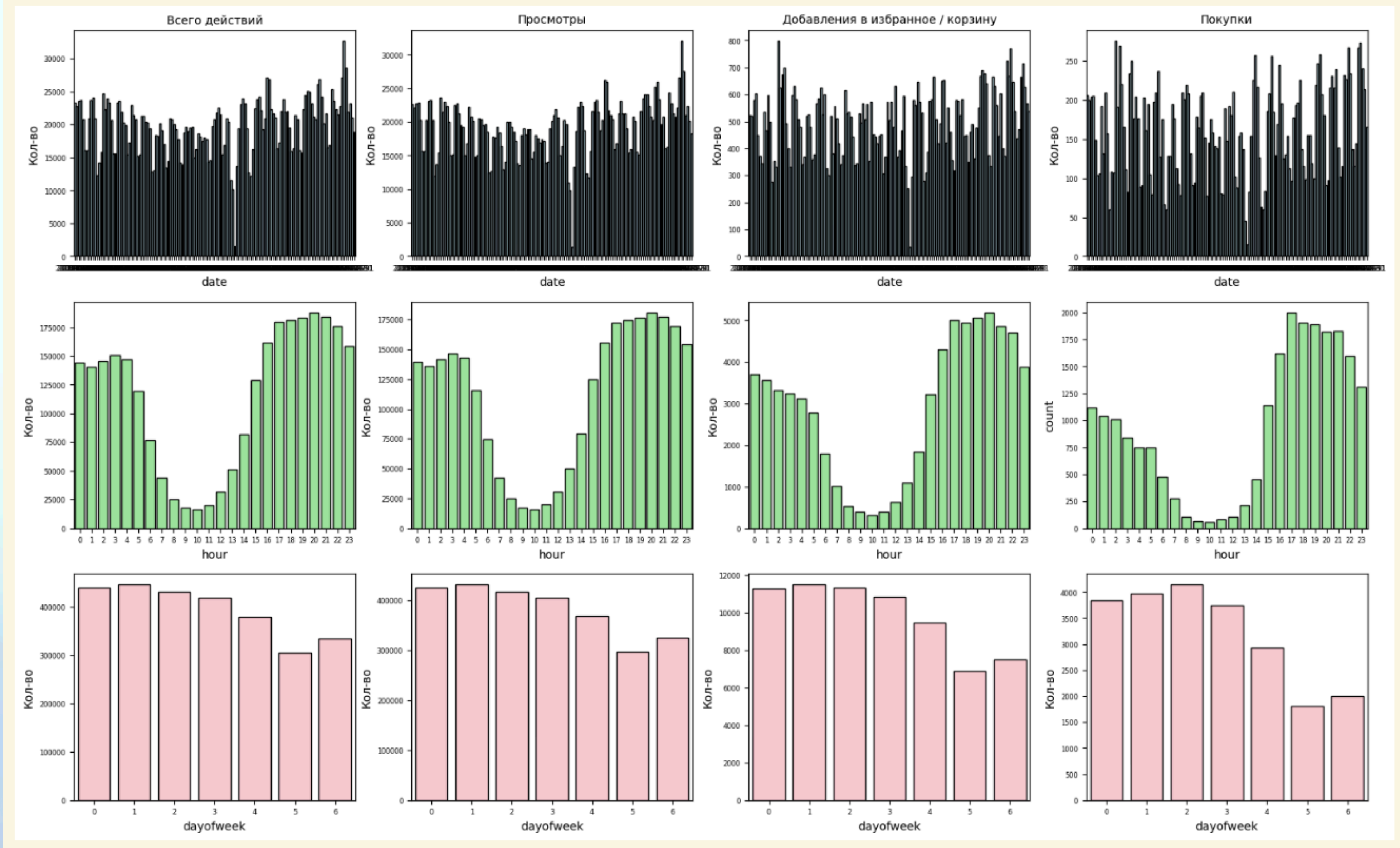
2. Файл дерева категорий содержит 1669 строк. Каждая строка в файле указывает дочерний CategoryID и соответствующий родительский.

3. Файл со свойствами элемента (item_properties.csv) содержит 20 275 902 строки, то есть различные свойства, описывающие 417 053 уникальных элемента. Файл разделен на 2 файла из-за ограничений по размеру файла. Поскольку свойство товара может меняться со временем (например, цена меняется с течением времени), каждая строка в файле имеет соответствующую временную метку. Другими словами, файл состоит из объединенных снимков за каждую неделю в файле с данными о поведении. Однако, если свойство элемента остается постоянным в течение наблюдаемого периода, в файле будет присутствовать только одно значение моментального снимка.

Файл свойств товара содержит столбец временных меток, поскольку все они зависят от времени, поскольку свойства могут меняться со временем, например цена, категория и т.д. Изначально этот файл состоял из снимков за каждую неделю в файле событий и содержал более 200 миллионов строк. Мы объединили последовательные значения постоянных свойств, поэтому он изменен из формы снимка в форму журнала изменений. Таким образом, постоянные значения появлялись бы в файле только один раз. Это действие значительно сократило количество строк в 10 раз.

Все значения в файле "item_properties.csv", за исключением свойств "categoryid" и "available", были хэшированы. Значение свойства "categoryid" содержит идентификатор категории элемента. Значение свойства "available" содержит доступность товара, т.е. 1 означает, что товар был доступен, в противном случае 0. Все числовые значения были отмечены символом "n" в начале и имеют точность в 3 цифры после запятой, например, "5" станет "n5.000", "-3.67584" станет "n-3.675". Все слова в текстовых значениях были нормализованы (процедура определения: <https://en.wikipedia.org/wiki/Stemming>) и хэшированы, числа были обработаны, как указано выше, например, текст "Hello world 2017!" станет "24214 44214 n2017.000"

Статистический анализ - действия

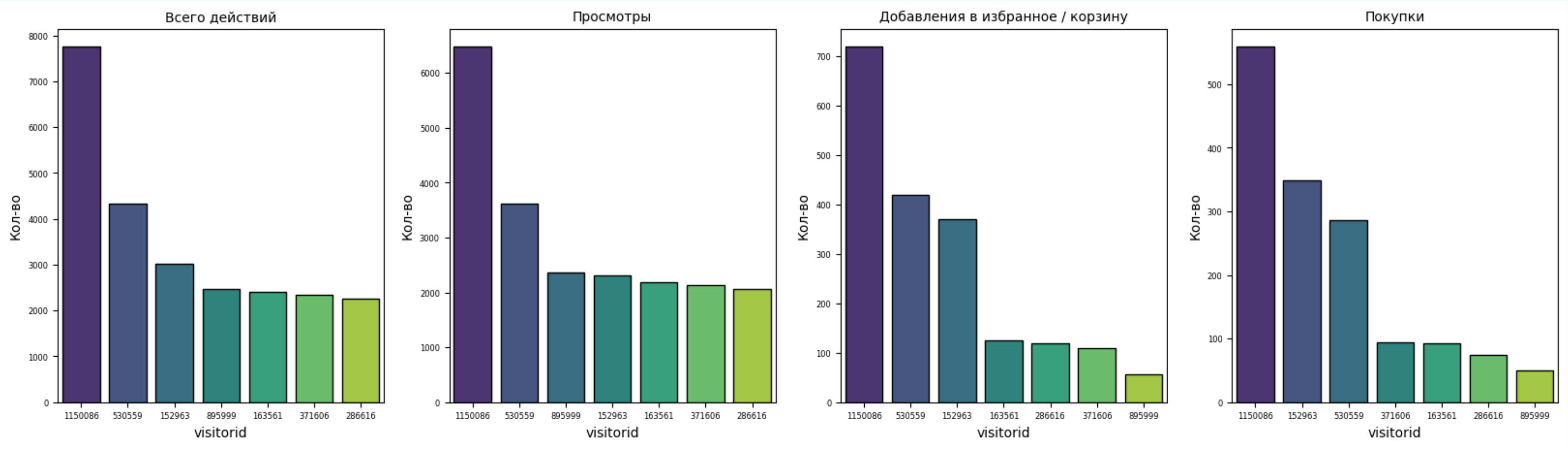


Действия клиентов в целом равномерны, наблюдается недельная / дневная сезонность.

В течение суток пик приходится на вечернее (после работы) время, спад на ночной время.

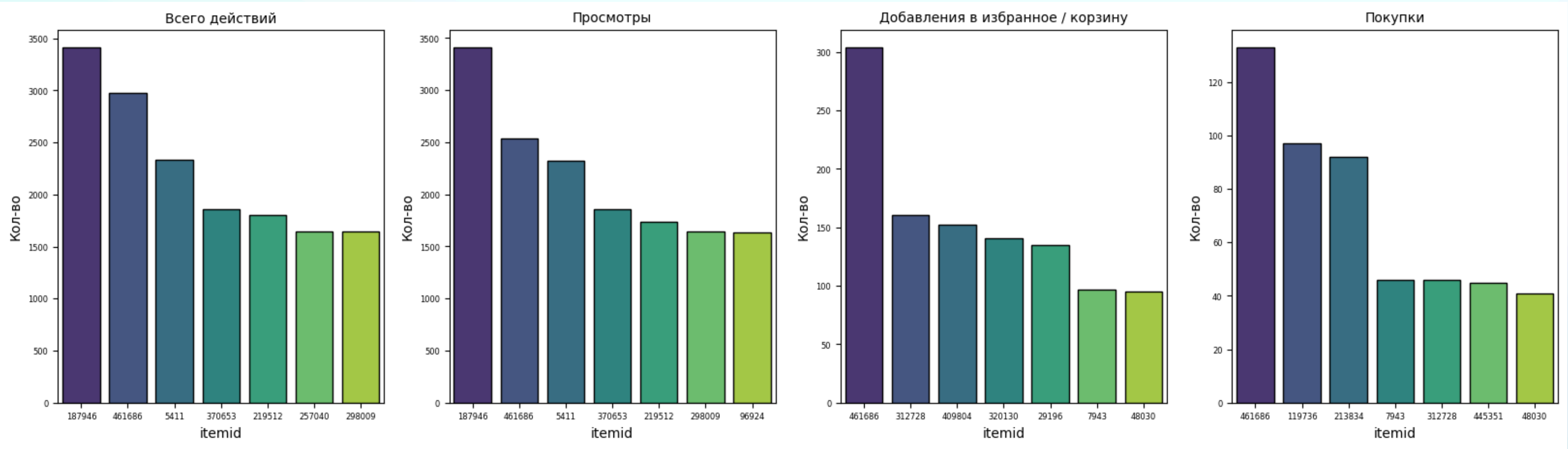
Максимальная активность в течение недели приходится на вторник - среду, спад в выходные дни.

Статистический анализ - клиенты / товары



Из графика мы видим, что самые активные клиенты в ТОП-7 остаются в списке по всем действиям, меняясь местами.

Клиент 1150086 - лидер во всех 3-х категориях действий.



Из графика мы видим, что ТОП-7 товаров меняется от действия к действию и самый просматриваемый товар 187946 отсутствует в 2-х других графиках.

Можно выделить товар 461686 как лидера по продажам и добавлениям в избранное.

Подготовка данных

В наших данных клиенты не дают явных оценок или предпочтений, то есть взаимодействие клиент - товар охарактеризовано действиями: просмотр / view, добавление в избранное (корзину) / addtocart, покупка / transaction. Мы можем предположить, что эта информация неявным образом отражает предпочтения пользователя в отношении товаров. В вариантах использования неявной обратной связи существует несколько методов для выполнения дедупликации, в зависимости от требований реальных бизнес-пользователей. Рассмотрим их и создадим данные с использованием этих методов:

1. В зависимости от бизнес задачи **экспертно** присваивать оценку типу взаимодействий, в нашем случае мы принимаем: просмотр / view = 0, добавление в избранное (корзину) / addtocart = 1, покупка / transaction = 1. Создаем новый признак с оценками и на преобразованных данных проводим моделирование;

2. Данные агрегируются пользователем для генерации некоторых оценок, представляющих предпочтения (в некоторых алгоритмах оценка называется affinity score по причине простоты, далее оценки называются affinity). Стоит отметить, что в таком случае баллы аффинити отличаются от рейтингов в явном наборе данных с точки зрения распределения значений. В этом случае алгоритм, используемый для обучения рекомендателя, должен быть тщательно подобран с учетом распределения баллов сходства, а не дискретных целых значений:

2.1. **Подсчет** количества взаимодействия между пользователем и товаром для получения оценок сходства;

2.2. **Взвешенный подсчет** - сочетает в себе элементы **экспертного** присвоения весов действий и агрегации данных путем сложения взаимодействия;

2.3. **Подсчет зависящий от времени** - во многих сценариях временная зависимость играет решающую роль при подготовке набора данных для построения модели совместной фильтрации, которая фиксирует изменение интересов пользователей с течением времени. Одним из распространенных методов получения счета, зависящего от времени, является добавление коэффициента затухания во времени при подсчете.

Для оценки работы моделей были сформированы 4 типа данных с учетом выше описанных методик:

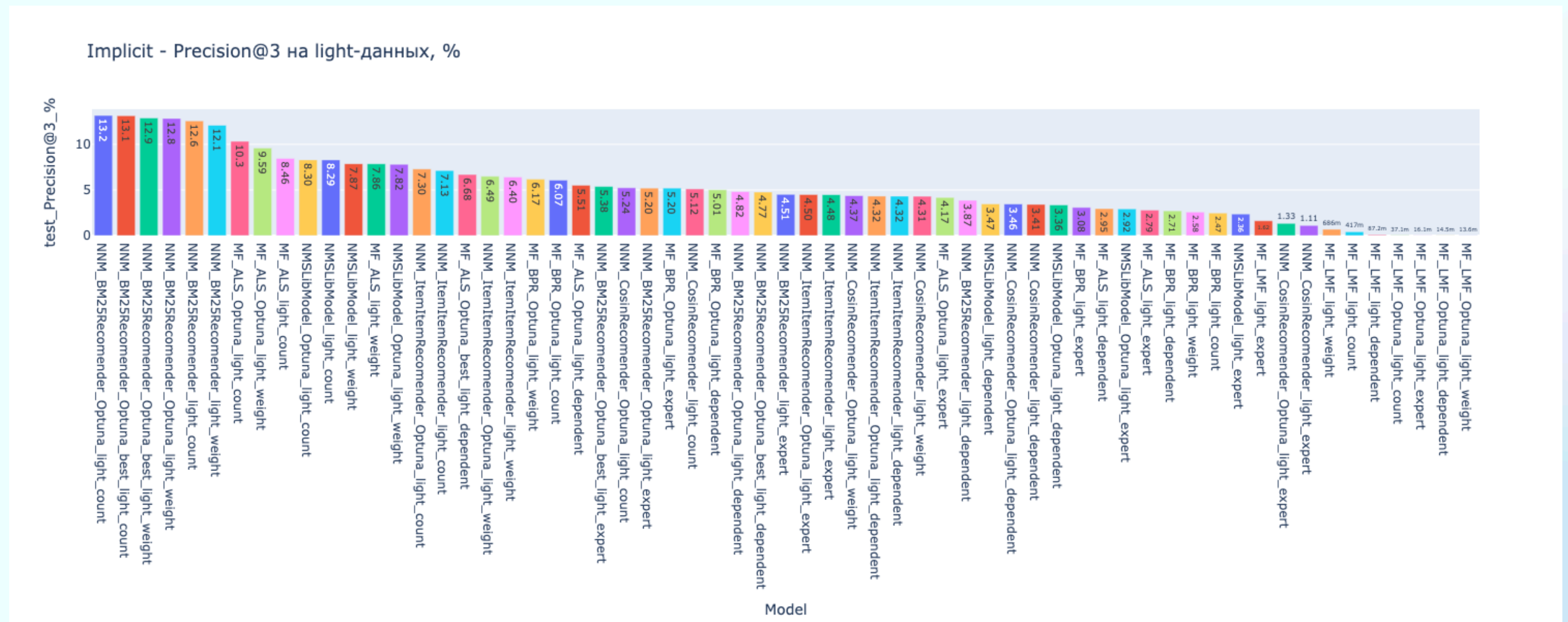
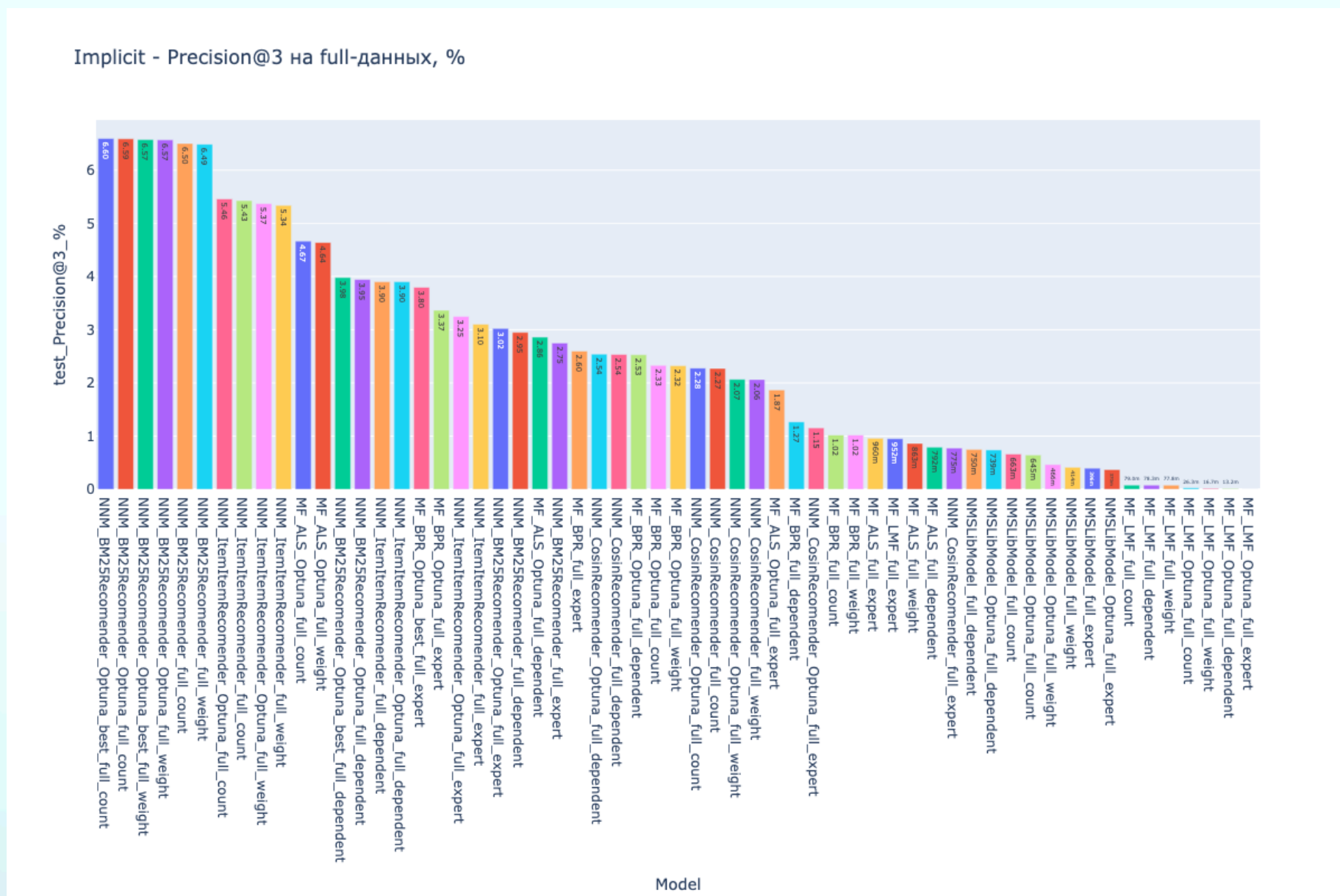
1. Expert - где покупке и добавлению в избранное / корзину присвоено значение - 1, просмотр - 0;
2. Count - количество взаимодействия клиент - товар подсчитаны;
3. Weight - взаимодействиям приданы веса: просмотр - 1, добавление в избранное / корзину - 3, покупка - 5 и взаимодействия посчитаны с учетом весов;
4. Dependent - взаимодействия weight с учетом времени взаимодействия, введен период затухания = 5.

Формирование объема выборки данных - для оценки влияния объема выборки данных на качество модели, данные были сформированы по следующему принципу:

1. Full - все наблюдения предоставленные заказчиком;
2. Light - исключены пользователи, которые имеют 5 и менее взаимодействий в выборке.

Библиотека Implicit - baseline

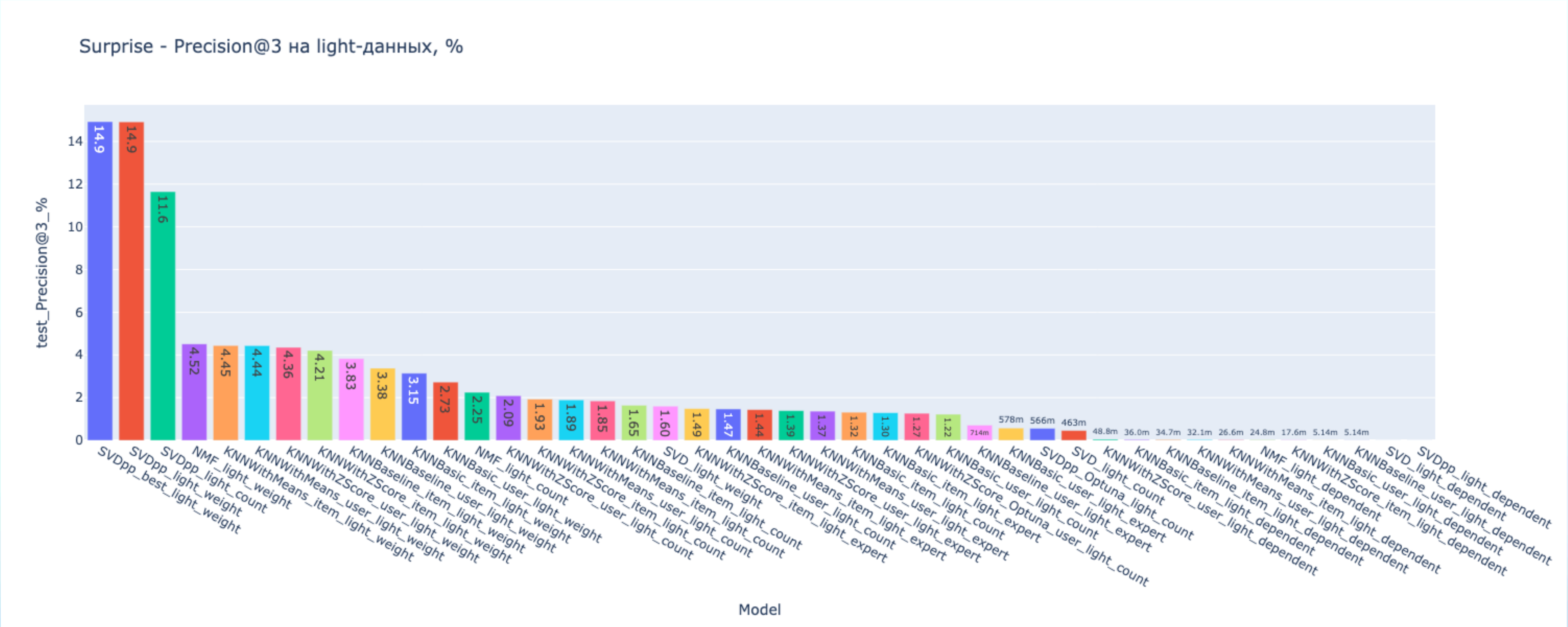
За базу возьмем библиотеку ****Implicit**** так как она заявлена как библиотека для работы с невыми наборами данных - «Быстрая Collaborative Filtering на Python для неявных наборов данных. Этот проект предоставляет быстрые реализации на Python нескольких различных популярных алгоритмов рекомендаций для неявных наборов данных обратной связи.



1. На сокращенных данных в библиотеке Implicit лучший результат по метрики и времени показали алгоритмы серии "Nearest Neighbour Models / Модели ближайших соседей" в частности:
 1. BM25Recommender - item-item рекомендация товара на расстоянии BM25 между товарами. BM25 использует весовую функцию, которую можно настроить на основе параметра K1. Изменяя значение K1 в этой функции, мы можем изменить форму, чтобы перейти от ступенчатой функции, используемой в расстоянии Жаккарда ($K1 = 0$), к линейному взвешиванию, используемому в косинусном расстоянии ($K1 = + \infty$);
 2. ItemItemRecomender - базовый алгоритм для рекомендательных моделей Item-ближайшего соседа. Лидер по времени обучения.
2. Лучшие результаты показаны при применении метода "Агрегирования данных" простым сложением количества взаимодействий (count) и взвешивания (weight).
3. При исключении пользователей с количеством взаимодействий 5 и менее, мы получили существенный прирост метрики с 6.6% до 13.2%.

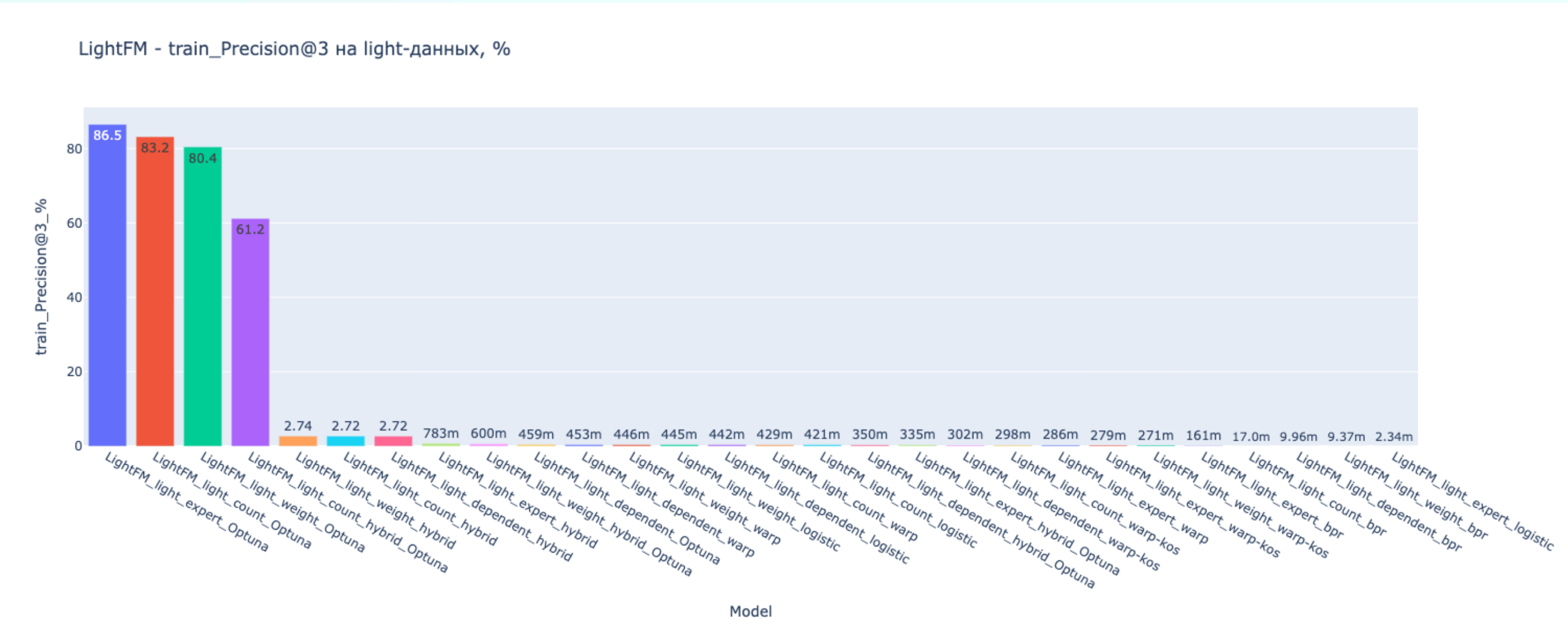
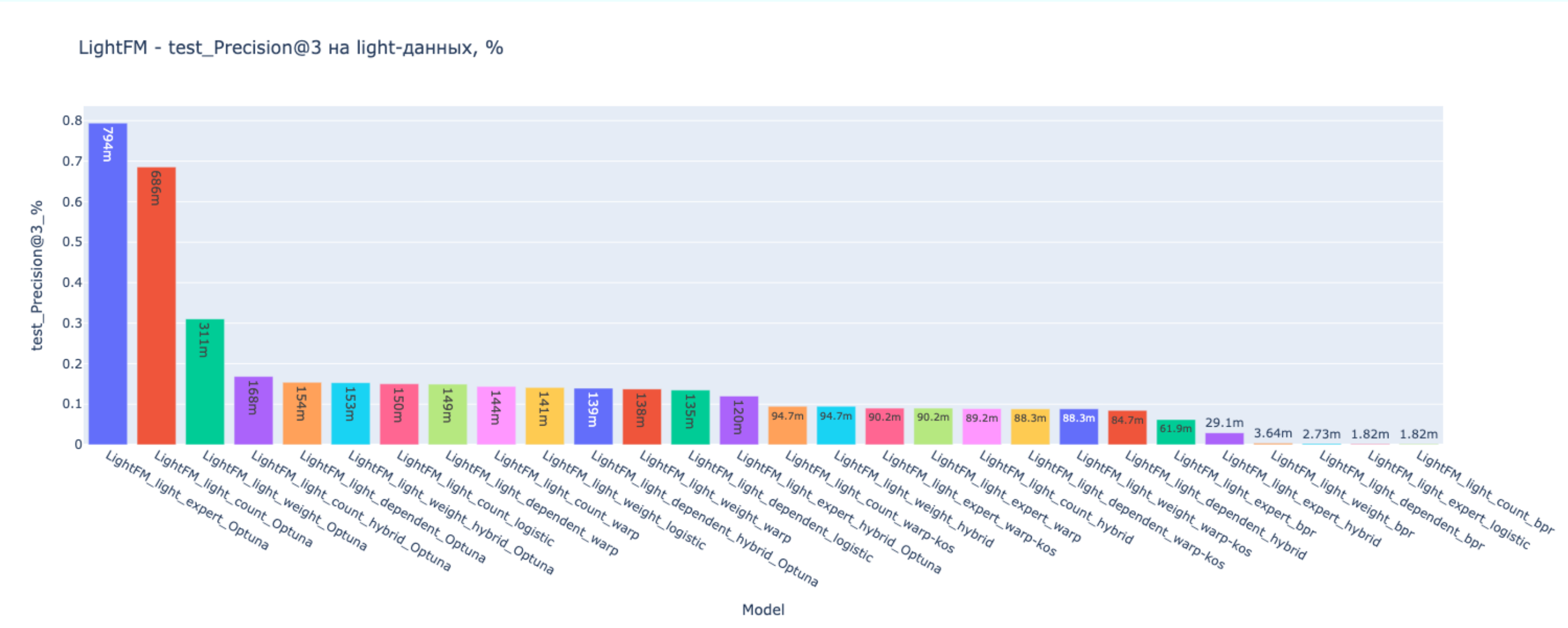
Библиотека Surprise

****Surprise**** — это библиотека Python, предназначенная для построения и оценки алгоритмов рекомендательных систем. Она особенно полезна для тех, кто занимается разработкой систем, основанных на методах коллаборативной фильтрации (Collaborative Filtering). Эта библиотека предоставляет множество готовых алгоритмов, а также инструменты для тестирования и сравнения их производительности.



- 1. На full данных работа алгоритмов библиотеки требует больших затрат по времени и ресурсам, оценку работы не производил. На light - отработала, но со значительно большим временем чем Implicit;
- 2. Лучший результат показал алгоритм SVDpp на взвешенных данных (weight), который составил 14,9%;
- 3. Необходимо обратить внимание, что в библиотеке отсутствует встроенная функция метрики Precision_at_k, но представлен пример кода для расчета и использования. При этом в функции есть настраиваемый параметр treshold, который необходимо настраивать в зависимости от данных. При работе на разных данных были выставлены параметры: weight / count = 3.5 (как указано в документации), expert - 0.5, и это может затруднить сравнение результатов работы алгоритмов с другими библиотеками.

Библиотека LightFM



****LightFM**** – это библиотека Python, разработанная специально для создания рекомендательных систем. Она предоставляет инструменты для обучения моделей, которые могут предсказывать предпочтения пользователей на основе их предыдущих взаимодействий с элементами системы (например, покупки товаров, просмотры фильмов и т.п.). Основная цель библиотеки – сделать процесс разработки рекомендательных систем простым и эффективным, предоставляя удобные интерфейсы для работы с данными и моделями.

LightFM может комбинировать разные подходы к созданию рекомендаций, такие как коллаборативная фильтрация и контентная фильтрация. Это помогает улучшить качество рекомендаций за счет использования различных источников информации. В своей работе оценим работу гибридной (hybrid) модели за счет использования сформированных данных property.

1. По результатам работы модели на базе LightFM мы види, что достигнут результат метрики Precision@3 = 86.5% на train выборке, но к сожалению на test результаты меньше 1%;
2. Так же следует отметить, что "Гибридная модель / Hybrid" не показала лучшего результата;
3. Лучший результат достигнут на данных light_expert.

Библиотека Recommenders

Библиотека ****Recommenders**** является мощным инструментом для создания рекомендательных систем. Она предлагает широкий спектр методов и алгоритмов, а также удобные средства для интеграции с другими библиотеками и источниками данных.

В notebook рассмотрена модель на основе алгоритма BiVAE, так как она одна из немногих моделей, которая, довольно легко «запустилась» и отработала. Так же в "Сравнительном анализе", который представлен на официальном сайте, она показала лучший результат по метрике Precision@k при проведении бенчмаркета моделей на данных Movielens и на 100K и на 1M.

К сожалению модель BiVAE не показала хороших первичных результатов на метрике Precision@3, а дальнейшая оптимизация параметров требует больших машинных ресурсов, что не позволило оценить данную модель в проекте. Так же необходимо отметить, что представленные алгоритмы в библиотеки довольно сложные и ресурсоемкие. Однако очень хорошо прописана документация и примеры.

Итоги

Оценка работы алгоритмов библиотек проводилась на данных, которых были разделены с помощью встроенных методов train-test-split (в основном это random) в библиотеках и оценка проводилась с помощью встроенных функций Precision_at_k.

Для качественной оценки работы алгоритмов из разных библиотек был проведен сравнительный анализ работы на данных разделенных по времени (Time-Based Split) в пропорции 90/10 и прямым сравнением рекомендаций и факта взаимодействия клиента и товаров. Параметры моделей использовались, которые были получены при подборе на предыдущем этапе. Методика сравнения следующая:

- * 3 рекомендованных товара сравнивались с полным фактическим списком товаров (просмотр, добавление в избранное, покупка) с которыми взаимодействовал клиент;
- * 3 рекомендованных товара сравнивались со списком товаров которые клиент добавил в избранное или купил.

По результатам данной оценки получен результат:

1 место - LightFM / LightFM_CF: 8.83% / 4,26% (все товары с которыми взаимодействовал клиент/ товары добавленные в избранное или купленные). Для полноценной оценки гибридной модели не хватило ресурсов машины;

2 место - Surprise / SVD++: 8.53% / 4,1% (все товары с которыми взаимодействовал клиент/ товары добавленные в избранное или купленные);

3 место - Implicit / BM25Recommender: 2.1% / 1.2% (все товары с которыми взаимодействовал клиент/ товары добавленные в избранное или купленные);

В соответствии с полученными результатами для построения финальной модели выбираем модель на базе алгоритма *LightFM (collaborative filtering)* библиотеки *LightFM*.

Для влияния деления данных на train и test была проведена оценка метрики на разных интервалах test (от 3 до 24 дня), которая показала:

1. Precision@3 на test = 3.0 дня составляет: 0.1874
2. Precision@3 на test = 7.0 дней составляет: 0.1546
3. Precision@3 на test = 10.0 дней составляет: 0.0887
4. Precision@3 на test = 14.0 дней составляет: 0.0426
5. Precision@3 на test = 17.0 дней составляет: 0.0305
6. Precision@3 на test = 21.0 день составляет: 0.0263
7. Precision@3 на test = 24.0 дня составляет: 0.0231

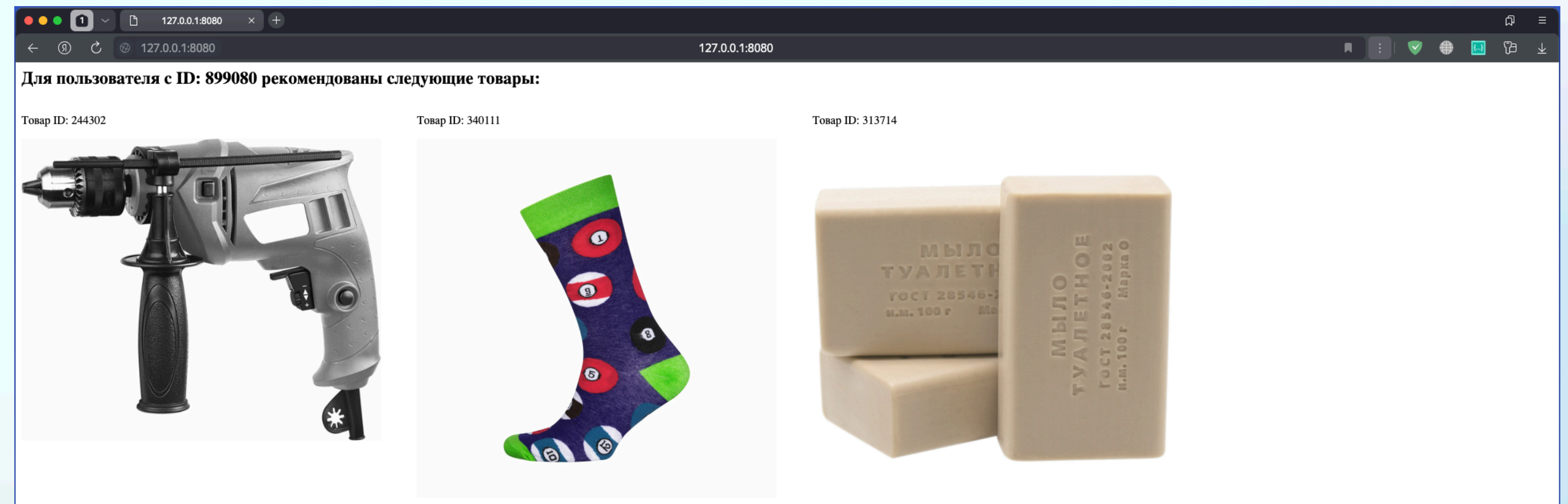
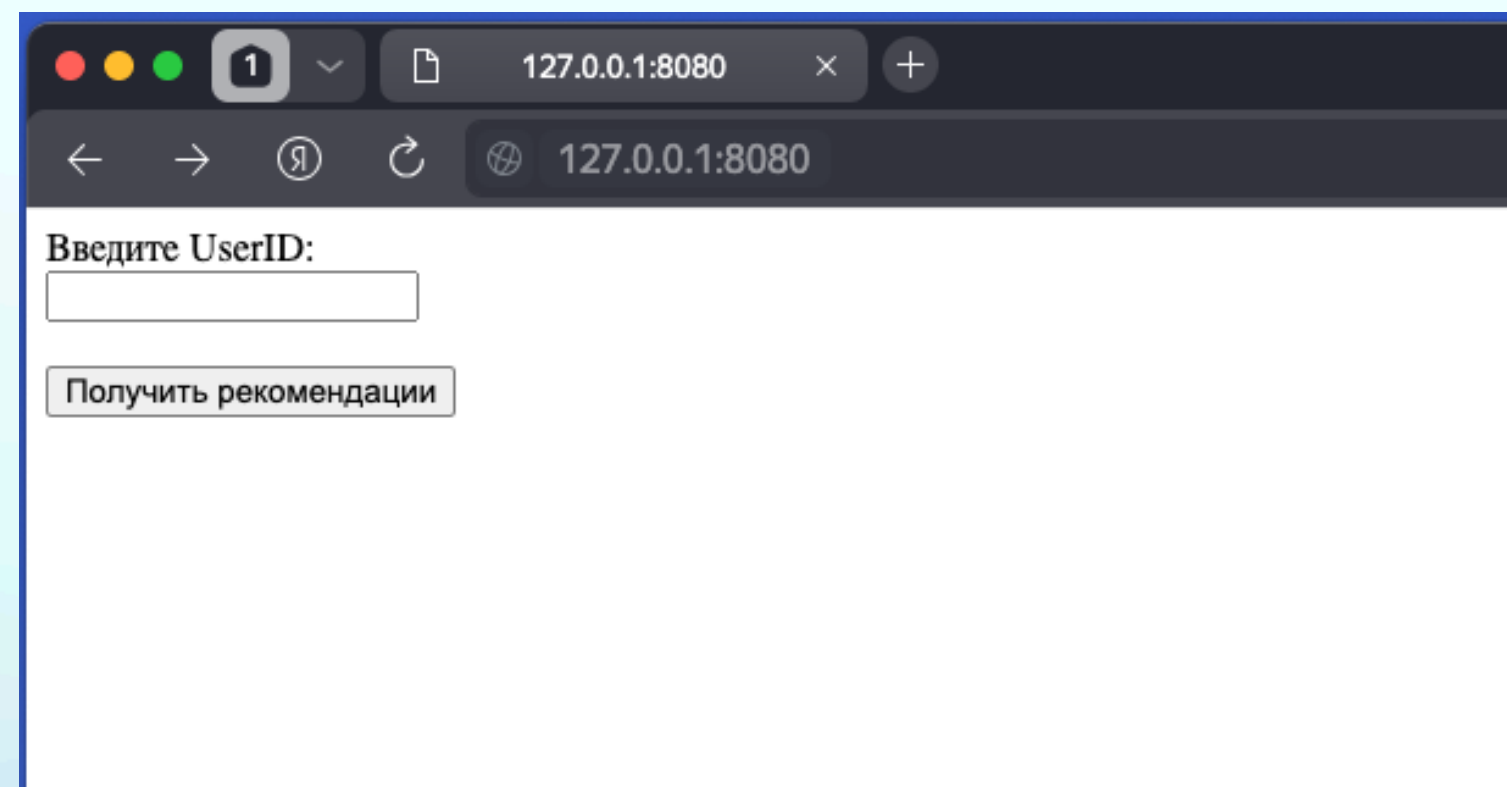
Мы видим, что снижение метрики почти в 2 раза происходит в интервале с 7 на 10 день. Полученный результат можно объяснить снижением интереса клиента к товарам, возможно он купил его или отказался от покупки. И как предположение - модель необходимо переобучать / дообучать не реже чем раз в 7 дней после внедрения.

Итоговая модель была обучена на полных данных. Результата Precision@3 = 79.31% (просмотр/избранное/покупка) / 47.74% (избранное/покупка).

Результаты / Материалы

Проверка работы модели:

1. Зайдите на Docker Hub по ссылке: https://hub.docker.com/repository/docker/avv1369/recommender_sistem/general
2. Загрузите образ на свой компьютер (объем 3.64+ GB, отдельно модель 2.2GB): `docker pull avv1369/recommender_sistem:latest`
3. В терминале или командной строке IDE запустите контейнер: `docker run -d -p 8080:5000 avv1369/recommender_sistem:latest`
4. Откройте браузер и введите адрес: 127.0.0.1:8080
5. В появившемся окне в поле «Введите UserID» - введите ID клиента (пример 899080), нажмите поле - получить рекомендации
6. Появится окно с рекомендациями для пользователя с фотографиями (фотографии выбираются случайным образом из 10 загруженных и для одного пользователя могут быть разными). Если ID клиента не представлен в модели, будет выведена надпись: «Клиент с ID отсутствует или для него нет рекомендаций».



Материалы по проекту представлены на следующих ресурсах:

1. GitHub (часть материалов) - https://github.com/avv1369it/Data_Science_School
2. Docker Hub (контейнер) - https://hub.docker.com/repository/docker/avv1369/recommender_sistem/general
3. Yandex disk (полный комплект) - <https://disk.yandex.ru/d/wLNiJnHJx-pMoQ>